

RuleGraph: A Legal Knowledge Graph with Executable Examination Logic

From Law-as-Code to a Subsumption-Capable Knowledge Net

RMS Engineering

Rule Mapping Solutions — Technical Report, May 2026

Abstract. Classical Law-as-Code projects compile individual statutory provisions into executable rule engines, yet they treat each norm in isolation and provide no structural account of how norms relate, what kind of legal proposition each one expresses, or how they are best located by a non-expert user. We present *RuleGraph*, a multi-model knowledge graph for German social law (SGB) that retains the executable character of Law-as-Code at the level of individual rule nodes while embedding every norm in a four-axis network of meaning: a structural hierarchy, an automatically detected reference graph, an embedding-based thematic clustering, and an expert-curated ontological classification down to branch granularity. On top of the graph we operate a tool-calling chatbot that interleaves retrieval and subsumption: it first localises the user's case (*where* to examine), then loads the applicable examination schema (*how* to examine), and finally subsumes the case under the relevant rule tree with formal AND/OR/XOR propagation. The result is a system that does not just retrieve norms but actively checks them, and that — in contrast to classical RAG — surfaces semantically and ontologically related norms across statute-book boundaries.

1. Introduction

The Law-as-Code paradigm models a legal provision as a Boolean or first-order expression over factual predicates: a single norm becomes a function from a case to a verdict. This is sufficient for the local question — *is the condition of § 7 SGB II satisfied?* — but it leaves three structurally important questions unanswered.

First, norms are not isolated objects; they cross-reference one another, often implicitly. A complete answer to a user's question typically requires reasoning over a small cluster of related norms, not a single one. Second, norms differ in *kind*: some confer entitlements, others impose prohibitions, define terms, or organise procedure. The proper order in which to examine the elements of a norm depends on this kind, and the kind is rarely encoded in the text itself. Third, users do not know which norm applies. A retrieval mechanism must be able to bridge from imprecise everyday language to the precise legal predicate. The local

question of one provision — *is the condition of § 7 SGB II satisfied?* — is therefore necessary but not sufficient.

RuleGraph addresses these three gaps by lifting the meta-layer that surrounds an individual norm into the data model itself. Every norm remains an executable rule tree (Law-as-Code at the leaf level), but the trees are interconnected through five edge types and embedded in two complementary cluster spaces: an empirical one, learned from text similarity and usage, and an ontological one, derived from a hand-curated legal schema. Retrieval traverses these layers in a defined three-stage protocol — *where, how, satisfied* — carried out by an LLM agent with a fixed tool vocabulary over the graph.

The contributions of this paper are (i) a formal schema for legal rule graphs that uniformly combines structural, referential, thematic, and ontological information (Section 3); (ii) a two-knowledge model that separates dynamic empirical clustering from a stable expert ontology (Section 4); (iii) a branch-level ontological classification procedure that maps individual RuleMap branches, not just statutory provisions, to their examination type (Section 8); and (iv) a three-stage retrieval-and-subsumption protocol with formal propagation rules (Sections 6–7).

2. Related Concepts

Three lines of work motivate the design. *Law-as-Code* in its modern form, as exemplified by systems such as Catala, OpenFisca, or earlier expert-system efforts, translates statute text into typed functional programs and supports unit tests against expected outcomes. These systems excel at the local execution of one norm but offer no structural account of inter-norm relations and rely on the user to know which provision to invoke.

The *knowledge graph* tradition in legal informatics, by contrast, focuses on relational structure — citations, definitions, applicability conditions — but historically stops short of executable rule logic. Graphs are navigated, not run. Recent work has begun to mix the two views; RuleGraph belongs to this strand and is, to our knowledge, the first to make the mixture systematic by attaching empirical and ontological cluster layers symmetrically.

Finally, *retrieval-augmented generation* (RAG) has become the de-facto pattern for grounding LLM answers in document corpora. Naive RAG retrieves passages by keyword or embedding similarity; this is insufficient for legal subsumption, which requires both (a) finding the correct provision and (b) following its internal logic. RuleGraph's retrieval protocol can be read as a structured generalisation of RAG in which the “retrieved” object is not a passage but an executable sub-tree, and the LLM acts as both retriever and subsumption engine.

3. Formal Schema

We model the legal corpus as a labelled multigraph

$$G = (V, E, \lambda, \sigma)$$

with a finite node set V , a finite edge multiset $E \subseteq V \times V$, a node-type labelling $\lambda: V \rightarrow \Lambda$, and an edge-property function $\sigma: E \rightarrow \mathcal{P}$, where $\mathcal{P}$ denotes attribute records (key–value maps).

The node set partitions into four disjoint subsets according to function:

$$V = V_p \cup V_s \cup V_c \cup V_\ell$$

where V_p are *examination nodes* (rule-tree nodes carrying logic), V_s are *structure nodes* (statutory hierarchy), V_c are *cluster nodes*, and V_ℓ are *legal sources* (norm texts).

3.1 Examination nodes

A node $v \in V_p$ is a 7-tuple

$$v = \langle \text{id}, \tau, \ell, q, \mathbf{k}, s, \mathbf{e} \rangle$$

where τ is a human-readable title, $\ell \in \{\wedge, \vee, \oplus\}$ is a logic type taken from conjunction, disjunction, and exclusive disjunction, q is the natural-language *query definition* that an evaluator must subsume against the case facts, $\mathbf{k}$ is a list of keywords, s is an LLM-generated summary, and $\mathbf{e} \in \mathbb{R}^d$ is an embedding with $d = 1536$ under the current encoder. The logic type composes the verdicts of v 's children into the verdict of v ; see Section 7.

3.2 Edges

Edges have five types, each with its own property record:

Edge type	Endpoints	Key properties
parent_of	$V_p \times V_p$	—
contains	$V_s \times V_s$	—
has_pruefung	$V_s \times V_p$	—
requires	$V_p \times V_p$	auto_detected
belongs_to	$(V_p \cup V_c) \rightarrow V_c$	edge_type, scope

The two semantic edge types `requires` and `belongs_to` are what makes the graph more than a tree of trees. A traversal can filter on edge properties to obtain precise sub-graphs — for instance, all ontological hypernodes related at full *scope* within a depth of three hops.

4. Two-Knowledge Model

The cluster set V_c is partitioned by `type` into two subsets, the *empirical* cluster set C_e and the *ontological* cluster set C_o :

$$V_c = C_e \cup C_o, \quad C_e \cap C_o = \emptyset.$$

The two subsets answer two different questions about a norm.

Empirical clusters answer the navigation question — *where in the corpus does this case belong?* They are produced by combining LLM-generated tags with K-Means over normalised embeddings and are subject to revision as the corpus grows. The current empirical layer contains 246 cross-reference clusters at level 2, 30 theme clusters, 8 examination-type clusters, and 5 legal-area clusters.

Ontological clusters answer the structural question — *what kind of legal proposition is this, and in what order must its elements be examined?* They are imported once from an expert-curated XML schema and consist of 94 nodes across 10 levels of depth, rooted in seven top-level categories: obligation, prohibition, entitlement, freedom of action, procedural law, definitions, and logical connectives.

A norm $n \in V_p$ is associated with both sets via the `belongs_to` relation. We write

$$\beta_e(n) = \{ c \in C_e \mid (n, c) \in E \},$$

$$\beta_o(n) = \{ c \in C_o \mid (n, c) \in E, \sigma(n, c).edge_type = \text{ontological_classification} \}.$$

The empirical attachment β_e is many-to-many and changes over time; the ontological attachment β_o is computed branch-by-branch (Section 8) and is, by construction, attached to the *deepest matching* hypernode of C_o . The ontology itself carries an internal hierarchy through `belongs_to` edges with `edge_type = ontological_hierarchy`; we treat C_o together with these edges as a rooted forest T_o .

5. Construction Pipeline

The graph is built bottom-up by a sequence of five idempotent transformations:

$$\emptyset \rightarrow_{T_1} G_1 \rightarrow_{T_2} G_2 \rightarrow_{T_3} G_3 \rightarrow_{T_4} G_4 \rightarrow_{T_5} G$$

where:

- **T_1 Import** parses RuleMap XML files and writes V_p , V_s , and the `has_pruefung` edges that link a statutory provision to its rule tree.
- **T_2 Annotation** scans norm texts for cross-references (e.g. § 7 in *Verbindung mit* § 9) and writes `requires` edges with `auto_detected = true`.
- **T_3 Clustering** imports the ontology from XML (yielding C_o and its hierarchy), and computes empirical clusters C_e by minimising

$$\arg \min_{C_1, \dots, C_k} \sum_{i=1}^k \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2,$$

- over embeddings $\mathbf{x} = \mathbf{e}_n$ of norm nodes, with cluster centroids $\boldsymbol{\mu}_i$ and a domain-tuned k .
- **T_4 Enrichment** uses an LLM to write a summary s , keyword list $\mathbf{k}$, and navigation hints for each node, and assigns the branch-level ontological label (Section 8).
- **T_5 Export** emits a flat JSON projection of the graph that the viewer consumes.

Each transformation is exposed as a CLI command of the form `rulegraph-pipeline <step>` and is safe to re-run.

5.1 Embedding similarity

Whenever a textual object is mapped to its embedding, similarity is computed by cosine:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

The current encoder is Azure OpenAI `text-embedding-ada-002`; the schema does not depend on the choice of encoder, only on its dimensionality d .

6. Three-Stage Retrieval

Given a user query q , retrieval proceeds in three semantic stages. Let $\mathbf{e}_q$ denote the embedding of q .

6.1 Stage 1 — Empirical: where?

Four sub-strategies run in parallel: a keyword scorer over V_p , a cluster scorer over C_e , a vector-search over both, and an ontological scorer over C_o . The vector-search uses thresholds $\theta_n = 0.70$ for nodes and $\theta_c = 0.65$ for clusters. The aggregated relevance of a candidate n for query q is a linear fusion of the four signals:

$$\text{score}(n, q) = \alpha \cdot \text{BM25}(n, q) + \beta \cdot \text{sim}(\mathbf{e}_q, \mathbf{e}_n) + \gamma \cdot \mathbb{1}[n \in \beta_e^{-1}(C^*(q))] + \delta \cdot \mathbb{1}[n \in \beta_o^{-1}(O^*(q))]$$

where $C^*(q)$ and $O^*(q)$ are the top-scoring empirical and ontological clusters under their respective scorers, and $\mathbb{1}[\cdot]$ is the indicator function. Top results are *auto-expanded*: the rule trees rooted at the top $k_n = 3$ nodes are rendered as compact ASCII trees with logic symbols $\wedge$, $\vee$, $\oplus$, and the top $k_c = 2$ clusters are expanded into summary plus example questions. The expanded context (~300–500 tokens) is injected into the agent's first prompt.

6.2 Stage 2 — Ontological: how?

The agent consults the ontological classification β_o of the candidate norm(s) to retrieve the applicable *examination schema*. The schema specifies the order in which the children of the rule-tree root should be evaluated and may dictate short-circuit behaviour beyond what the logic type alone implies. For a norm n classified into the entitlement subtree, for instance, the schema fixes the precedence of entitlement-creating versus entitlement-excluding elements.

6.3 Stage 3 — Subsumption: satisfied?

The agent now traverses the rule tree of the selected norm and evaluates its nodes against the case facts $\mathcal{F}$, a key–value store of user-supplied answers. The evaluation function is defined recursively in Section 7. Whenever $\mathcal{F}$ is insufficient to decide a leaf, the agent suspends and asks the user a bundled follow-up question; the user's answer is appended to $\mathcal{F}$ and execution resumes.

7. Subsumption Engine

Let $\mathbb{T} = \{\top, \perp, ?\}$ be the verdict lattice with truth, falsity, and an “undecided” value for unevaluated nodes. We define the evaluation function

$$\text{eval}: V_p \times \mathcal{F} \rightarrow \mathbb{T}$$

recursively. For a leaf v , the verdict is produced by the LLM acting as a subsumption oracle over the query definition $q(v)$ and the case facts:

$$\text{eval}(v, \mathcal{F}) = \text{LLM}(q(v), \mathcal{F}) \quad \text{if } \text{children}(v) = \emptyset.$$

For an inner node, the verdict is the lattice composition of the children's verdicts under the node's logic type $\ell(v)$:

$$\text{eval}(v, \mathcal{F}) = \begin{cases} \bigwedge_{c \in \text{children}(v)} \text{eval}(c, \mathcal{F}) & \text{if } \ell(v) = \wedge, \\ \bigvee_{c \in \text{children}(v)} \text{eval}(c, \mathcal{F}) & \text{if } \ell(v) = \vee, \\ \bigoplus_{c \in \text{children}(v)} \text{eval}(c, \mathcal{F}) & \text{if } \ell(v) = \oplus, \end{cases}$$

where the lattice operations extend the Boolean ones to handle $?$ by propagating it as unknown wherever it cannot be eliminated by short-circuit.

7.1 Short-circuit propagation

Two short-circuit rules eliminate unnecessary evaluation work and limit the number of questions posed to the user. Let $c \in \text{children}(v)$:

$$\ell(v) = \wedge \quad \wedge \quad \text{eval}(c, \mathcal{F}) = \perp \quad \Rightarrow \quad \text{eval}(v, \mathcal{F}) = \perp,$$

$$\ell(v) = \vee \quad \wedge \quad \text{eval}(c, \mathcal{F}) = \top \quad \Rightarrow \quad \text{eval}(v, \mathcal{F}) = \top.$$

These rules apply *upward* after every leaf evaluation: as soon as a leaf produces $\perp$ under a conjunction, the conjunction's verdict and, transitively, those of its conjunctive ancestors become $\perp$ without inspecting the remaining children. The procedure terminates because each evaluation step either resolves a strictly smaller subtree or short-circuits it entirely.

7.2 Case facts

The fact store grows monotonically across turns. After every leaf that produces $?$ for lack of information, the agent emits a bundled `ask_user` call, the answer is appended to $\mathcal{F}$, and the leaf is re-evaluated. A hard cap of four `ask_user` rounds prevents unbounded interrogation; after the cap the agent must work with the existing facts or answer with uncertainty.

8. Branch-Level Classification

A naive classifier maps an entire statutory provision to a single ontological category. This is inadequate because a single provision — for instance, § 31 SGB II — mixes obligations, exclusion grounds, and procedural elements in different branches of the same rule tree. RuleGraph instead classifies *every* branch of the tree.

Formally, let n be a norm with rule tree $R(n)$ rooted at $r_n \in V_p$. The classification procedure produces a partial function

$$\beta_o^{branch}: V_p \cap R(n) \rightarrow C_o$$

that assigns each relevant branch node to its matching ontological hypernode, choosing the deepest match available (Option 3 in our design notes). The classification is realised by an LLM that sees the full rule tree with node identifiers and emits a list of (node-id, ontology-id, scope) triples; for § 31 SGB II this yields 43 mappings.

This branch-level granularity has two practical consequences. First, retrieval can target a specific examination type without retrieving the entire provision — for instance, only the entitlement-excluding branches of a given norm. Second, traversal of the ontological hierarchy from a leaf upward yields a precise “legal flavour” for any single rule node, which the agent uses to choose the right examination schema in Stage 2.

9. Worked Example

Consider the query “*Am I entitled to Bürgergeld if I am a single parent and my child is in daycare?*”

Stage 1. The pre-search produces, in parallel: keyword hits § 7 SGB II (eligibility) and § 21 SGB II (additional needs); empirical-cluster matches *single parents* and *Bedarfgemeinschaft*; an embedding match on § 22 SGB VIII (daycare support), which shares no surface tokens with the query but is semantically close; and incoming **requires** edges pulling in neediness and cooperation norms.

Stage 2. The agent calls **get_ontological_schema** on § 7 and § 21. Both return an examination schema of category *Anspruch* (entitlement), specifying that entitlement-creating elements precede entitlement-excluding ones.

Stage 3. The agent identifies open elements in the rule tree of § 7 — age of the child, presence of other adults, own income — and emits a single bundled **ask_user** call. Once $\mathcal{F}$ is populated, leaf evaluations proceed; the propagation rules yield $\top$ at the root of § 7 and at the root of the additional-needs branch of § 21. The agent answers that both apply and that the daycare support of § 22 SGB VIII constitutes a separate, parallel claim — an answer that a keyword-only retriever would have failed to compose, because § 21 and § 22 SGB VIII do not occur in the surface form of the user's question.

10. Current State

At the time of writing the graph contains five empirical cluster layers with 246 cross-reference clusters at level 2, 30 theme clusters, 8 examination-type clusters, and 5 legal-area clusters, alongside an ontological forest of 94 nodes across 10 depth levels imported from `clustering_fuer_graphen_607.xml`. Embeddings are 1536-dimensional. Branch-level classification has been applied across the SGB II corpus; the example provision § 31 SGB II carries 43 branch-to-ontology mappings.

The retrieval system runs as a Streamlit application with an asynchronous AQL backend against ArangoDB. The agent loop is capped at thirty iterations and four user-clarification rounds per session; in practice the typical session converges in five to ten iterations.

11. Future Work

The current agent *sees* the retrieved context but does not actively reason about its own position in the graph. The propagation logic of Section 7 produces an implicit position — the set of evaluated nodes and their verdicts — but this is not exposed to the LLM as a first-class object. The next iteration introduces three capabilities:

Position tracking. An explicit data structure that records the current empirical cluster, the current ontological hypernode, and the active branch of the rule tree, refreshed after every tool call and presented to the LLM as a structured prompt fragment.

Situational norm pull-in. When the position tracker enters a sub-area that is known to interact with a different statute book — for instance, child-related provisions of SGB II interacting with SGB VIII — the agent automatically issues a follow-up retrieval over the corresponding ontological branch.

Introspection. A tool that lets the agent ask itself why it is asking a particular question. Operationally, this consists of presenting the unevaluated siblings of the deepest still-undecided node together with their logic type, so that the agent can choose whether to continue downward or to abandon the branch under a short-circuit.

Together these three additions promote the implicit graph position into a tractable first-class state, which we expect to reduce the average number of user clarifications per session and to enable cross-statute-book reasoning without manual prompting.

12. Conclusion

RuleGraph is an attempt to generalise Law-as-Code without giving up its executable character. Each individual norm remains a typed rule tree that subsumes a case to a verdict under transparent propagation rules. The novelty lies in the layers that surround the trees:

structural edges, automatically detected references, an embedding-based empirical clustering that grows with the corpus, and a stable expert ontology against which every branch of every rule tree is classified. A three-stage retrieval protocol — *where, how, satisfied* — runs over this multi-layer graph and is mediated by an LLM with a fixed tool vocabulary. The system retrieves and examines rather than retrieving alone, and it composes answers across statute books that a flat keyword or embedding retriever would split apart.

References

1. D. Merigoux, N. Chataing, J. Protzenko. *Catala: A Programming Language for the Law*. Proc. ACM Program. Lang. 5 (ICFP), 2021.
2. OpenFisca Project. *OpenFisca: Software to compute legislation*. Online: openfisca.org.
3. L. T. McCarty. *Reflections on TAXMAN: An Experiment in Artificial Intelligence and Legal Reasoning*. Harvard Law Review, 1977.
4. T. Bench-Capon, F. P. Coenen, P. Visser. *Knowledge based systems for legal applications*. Artificial Intelligence Review, 1993.
5. P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS, 2020.
6. S. Hogan et al. *Knowledge Graphs*. ACM Computing Surveys, 2021.
7. ArangoDB. *Multi-Model Database Documentation*. Online: docs.arangodb.com.
8. OpenAI. *text-embedding-ada-002 model card*. Online: platform.openai.com.